

UniAP: Unifying Inter- and Intra-Layer Automatic Parallelism by Mixed Integer Quadratic Programming

Hao Lin*, Ke Wu*, Jie Li*, Jun Li, Wu-Jun Li†

National Key Laboratory for Novel Software Technology
 School of Computer Science

Nanjing University, Nanjing 210023, China

{hao.lin, ke.wu, jie-li, lijun}@smail.nju.edu.cn, liwujun@nju.edu.cn

Abstract

Distributed learning is commonly used for training deep learning models, especially large models. In distributed learning, manual parallelism (MP) methods demand considerable human effort and have limited flexibility. Hence, automatic parallelism (AP) methods have recently been proposed for automating the parallel strategy optimization process. Existing AP methods suffer from sub-optimal solutions because they do not jointly optimize the two categories of parallel strategies (i.e., inter-layer parallelism and intra-layer parallelism). In this paper, we propose a novel AP method called UniAP, which unifies inter- and intra-layer automatic parallelism by mixed integer quadratic programming. To the best of our knowledge, UniAP is the first parallel method that can jointly optimize the two categories of parallel strategies to find an optimal solution. Experimental results show that UniAP outperforms state-of-the-art methods by up to $3.80\times$ in throughput and reduces strategy optimization time by up to $107\times$ across five Transformer-based models.

1. Introduction

Distributed learning (also called parallel learning) on clusters with several machines or GPUs is commonly used for training deep learning models, especially for some large models with billions of parameters [1, 40, 41]. Several parallel strategies, including pipeline parallelism (PP), data parallelism (DP), tensor parallelism (TP), and fully sharded data parallelism (FSDP), have been proposed for distributed learning. These parallel strategies can be divided into two main categories: inter-layer parallelism and intra-layer parallelism. Inter-layer parallelism [7, 9, 10, 15, 20, 21, 28, 29], which includes PP, partitions the model into disjoint sets

without partitioning tensors in each layer. Intra-layer parallelism [8, 22, 30, 34], which includes DP, TP, and FSDP, partitions tensors in a layer along one or more axes.

The parallel method¹ in one specific distributed learning method or system typically adopts one parallel strategy or a combination of several parallel strategies. Existing parallel methods can be divided into two categories: manual parallelism (MP) methods and automatic parallelism (AP) methods. In MP methods [30, 37, 44], one or several parallel strategies are manually optimized by researchers or developers. MP methods require extensive domain knowledge in deep learning models and hardware architectures. With the rapid development of deep learning models and the increasing diversity of modern hardware architectures [11, 12], MP methods demand considerable human effort and have limited flexibility.

To address the two limitations of MP methods, AP methods [14, 28, 47] have recently been proposed for automating the parallel strategy optimization process. Although existing AP methods have achieved promising progress, they optimize the two categories of parallel strategies separately rather than jointly. More specifically, some methods optimize only one category of parallel strategies [2, 16, 17, 23, 36, 42, 46], and the others optimize inter- and intra-layer parallelism hierarchically [9, 14, 28, 38, 39, 47]. Hence, existing AP methods suffer from sub-optimal solutions.

In this paper, we propose a novel AP method called UniAP for distributed learning. The contributions of UniAP are outlined as follows:

- UniAP unifies inter- and intra-layer automatic parallelism by mixed integer quadratic programming (MIQP) [19].
- To the best of our knowledge, UniAP is the first parallel method that can optimize the two categories of parallel strategies jointly rather than separately to find an optimal solution.

*Equal contribution.

†Corresponding author.

¹To avoid confusion, we treat ‘parallel method’ and ‘parallel strategy’ as two different terminologies in this paper.

- Experimental results show that UniAP outperforms state-of-the-art methods by up to $3.80\times$ in throughput and reduces strategy optimization time by up to $107\times$ across five Transformer-based models.

2. Background

2.1. Parallel strategy

Pipeline parallelism (PP) In PP [15], each worker (machine or GPU) holds a subset of model layers. Adjacent layers on different workers need to transfer activations in the forward propagation (FP) step and gradients in the backward propagation (BP) step.

Data parallelism (DP) In DP [22], each worker holds a replica of the whole model and partitions training samples. In each iteration, each worker computes gradients and synchronizes them with the other workers using all-reduce collective communication (CC). All workers will have the same model parameters after the synchronization step.

Tensor parallelism (TP) In TP [30], each worker holds a replica of training samples and partitions within model layers. In each iteration, each worker computes its local outputs in FP and its local gradients in BP. To synchronize outputs and gradients, all workers will perform all-reduce CC in FP and BP steps according to the partition scheme.

Fully sharded data parallelism (FSDP) FSDP [8, 32] partitions optimizer states, parameters and gradients of the model into separate workers. During the FP and BP step of each iteration, FSDP performs an all-gather CC to obtain the complete parameters for the relevant layer, respectively. After computing gradients, FSDP conducts a reduce-scatter CC to distribute the global gradients among the workers.

2.2. Parallel method

Manual parallelism (MP) MP refers to the parallel methods in which human experts design and optimize the parallel strategies. Representative MP methods include Megatron-LM [30], Mesh-TensorFlow [37], and GSPMD [44]. Megatron-LM manually designs TP and PP strategies for training Transformer-based models and exhibits superior efficiency. Mesh-TensorFlow and GSPMD require human effort to designate and tune the intra-layer parallel strategy. These methods rely on expert design and have little flexibility, challenging their automatic application to other models.

Inter-layer-only AP or intra-layer-only AP For inter-layer-only AP, GPipe [15] and vPipe [46] employ a balanced partition algorithm and a dynamic layer partitioning middleware to partition pipelines, respectively. For intra-layer-only AP, OptCNN [16], TensorOpt [2], and Tofu [42] employ dynamic programming methods to optimize DP and TP strategies together. FlexFlow [17] and Automap [36] use the Monte Carlo method to find the optimal DP and TP strat-

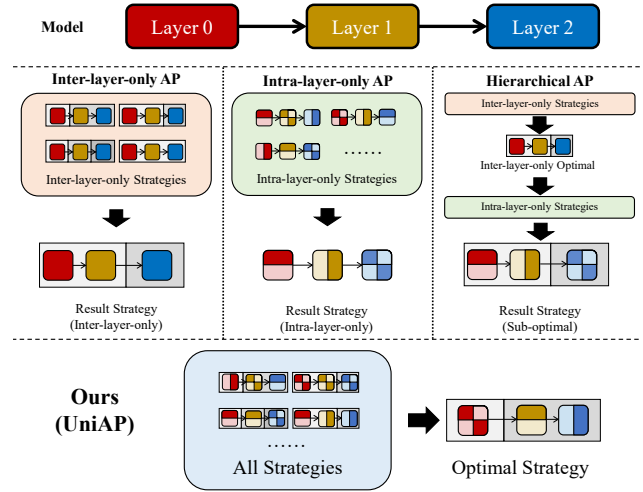


Figure 1. Parallel methods for optimizing parallel strategies for a three-layer model. The different arrangements of slices with varying transparency within the same layer block indicate different intra-layer parallelism strategies adopted by layers. The different arrangements of gray blocks which wrap the layer blocks indicate different inter-layer parallelism strategies.

egy. Colossal-Auto [23] utilizes integer programming (IP) techniques to generate intra-layer parallelism and activation checkpointing strategies. All these methods optimize only one category of parallel strategies.

Inter- and intra-layer AP PipeDream [28], DAPPLE [9], and PipeTransformer [14] use dynamic programming to determine optimal strategies for both DP and PP. DNN-partitioning [38] adopts IP and dynamic programming to explore DP and PP strategies. DT-FM [45] combines genetic algorithm and the local search strategy to explore DP and PP strategies in a layer-wise manner. Piper [39] and Alpa [47] adopt a parallel method considering DP, TP, and PP. Galvatron [25] uses dynamic programming to determine DP, TP, and FSDP strategies in a single pipeline stage. As for PP, it partitions stages and determines micro-batch size using naive greedy algorithms. All these methods are hierarchical, which will result in sub-optimal solutions.

3. Method

In this section, we introduce our proposed method called UniAP, which jointly optimizes the two categories of parallel strategies, including PP, DP, TP, and FSDP, to find an optimal solution. Figure 1 illustrates the difference between UniAP and other automatic parallelism methods. Inter-layer-only and intra-layer-only AP methods optimize (search) from a set of candidate inter-layer-only and intra-layer-only parallel strategies, respectively. Hierarchical AP methods first adopt greedy or dynamic programming to propose candidate inter-layer parallel strategies. Then, they optimize the intra-layer parallel strategy for every fixed inter-layer parallel strategy.

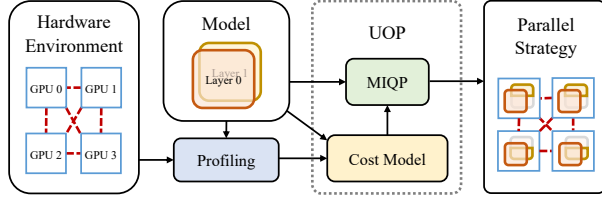


Figure 2. Flowchart of UniAP.

UniAP has the largest strategy space for exploration (joint optimization).

Figure 2 illustrates the flowchart of UniAP. UniAP first profiles the runtime information for the user’s hardware environment and the deep learning model. After that, UniAP estimates inter- and intra-layer costs given the computation graph and profiling results with its cost models. The estimated costs and the computation graph are then transformed into an MIQP problem. The objective function of the MIQP is to maximize the training throughput, or in other words, to minimize the training time per iteration (TPI). By iteratively applying the cost model and MIQP with different parameters, UniAP determines the minimal TPI and its corresponding parallel strategies. We name this process the Unified Optimization Process (UOP). Finally, UniAP interprets the parallel strategies into the execution plan for the designated model.

3.1. Profiling

UniAP collects runtime information about the hardware environment and deep learning model during profiling. For the hardware environment, UniAP evaluates the efficiency of all-reduce and point-to-point (P2P) communication for different device subsets. For example, when profiling a node with 4 GPUs, UniAP measures the all-reduce efficiency for various DP, TP, and FSDP combinations across these GPUs. Additionally, UniAP ranks these GPUs from 0 to 3 and evaluates the speed of P2P for two pipeline options: (0 → 2 and 1 → 3) and (0 → 1, 1 → 2 and 2 → 3). Furthermore, UniAP estimates the computation-communication overlap coefficient (CCOC) [25, 33].

UniAP acquires two types of information for the deep learning model: computation time and memory usage. On one hand, UniAP distinguishes the forward computation time per sample for different types of hidden layers. On the other hand, UniAP collects memory usage information for each layer, including the memory occupied by parameters and the memory usage of activation per sample in different TP sizes.

3.2. Cost model

UniAP employs two primary cost models, namely the time cost model and the memory cost model.

Time cost model To estimate computation time, UniAP first calculates the forward computation time by multiplying the batch size with the forward computation time per sample obtained from profiling. Users can obtain a more precise result by profiling the forward time on the specified batch size. For Transformer-based models that mainly consist of the MatMul operator, the computation time in the BP stages is roughly twice that of the FP stages [21, 25, 30]. Additionally, UniAP estimates the communication time by dividing the size of transmitting tensors by the profiled communication efficiency for different communication primitives. To accommodate overlapping, UniAP multiplies the profiled CCOC by the overlapping interval of computation and communication. To model the communication time between pipeline stages, UniAP calculates the cross-stage cost between consecutive stages by the summation of P2P costs.

Memory cost model UniAP estimates memory consumption for each layer with its memory cost model. This estimation consists of three steps for a given layer. First, it computes the activation memory cost m_a by multiplying the batch size and the profiled activation memory cost per sample of the TP size used by the strategy. Next, UniAP calculates the memory cost of model states m_s for each layer based on their parameter size ps , TP size ts , FSDP size fs , and a constant c_{dtype} dependent on the data type. Formally, we have

$$m_s = \frac{c_{dtype} \times ps}{ts \times fs}. \quad (1)$$

For example, if we choose precision as FP32, then $c_{dtype} = (4 + 4 + 4 + 4)/4 = 4$ since the learnable parameters, gradients, momentum and variance consume equal size of memory. If we opt for mixed precision with FP16 activated, then $c_{dtype} = (4 + 4 + 4 + 2 + 2)/2 = 8$. Finally, UniAP aggregates the activation memory cost m_a , memory cost of model states m_s , and context memory cost m_c to a constant matrix $\mathbf{M}$, where $\mathbf{M}_{uk}$ denotes the memory cost for the k -th intra-layer strategy of layer u on a single device.

3.3. Mixed integer quadratic programming

The estimated costs and the computation graph are then transformed into an MIQP problem. Its formulation includes an objective function and several constraints.

3.3.1. Objective function

The objective function tries to minimize TPI. In this paper, we have chosen GPipe as our PP strategy for illustration.² Figure 3 depicts the time cost decomposition of a GPipe-style PP with non-negligible communication costs. The time needed to apply gradients at the end of each iteration is not included, as it depends on the optimizer and is insignificant compared to the total time spent on FP and BP.

²UniAP is also compatible with other PP strategies. For example, users need to modify only the memory constraint in Section 3.3.2 to adapt to synchronous 1F1B pipeline [9, 29].

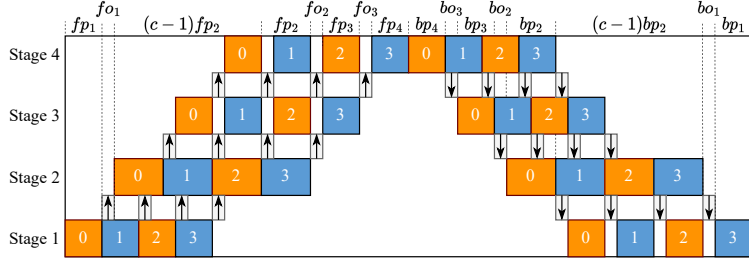


Figure 3. Time cost decomposition of a GPipe-style PP.

We denote the cost for computation stages as $\mathbb{P} = \{p_1, p_2, \dots, p_{deg}\}$ and the cost for communication stages as $\mathbb{O} = \{o_1, o_2, \dots, o_{deg-1}\}$. Here, deg represents the number of computation stages, which corresponds to the pipeline parallel size. In Figure 3, fp_i and bp_i denote forward and backward computation time for computation stage i , respectively. fo_j and bo_j denote forward and backward communication time for communication stage j , respectively. Hence, we have $p_i = fp_i + bp_i$ and $o_j = fo_j + bo_j$.

In a GPipe-style pipeline, we use c to denote the number of micro-batches. As illustrated in Figure 3, a mini-batch is uniformly split into four micro-batches, and the total TPI is determined by the latency of all computation and communication stages and the latency of the slowest stage. We further denote TPI in GPipe as $tpi_{gpipeline}$. Given that a stage with a higher FP computation cost leads to a higher BP computation cost with high probability, we can write the objective function of GPipe-style pipeline as follows:

$$\min tpi_{gpipeline} = \sum_{i=1}^{deg} p_i + \sum_{j=1}^{deg-1} o_j + (c-1) \max(\mathbb{P} \cup \mathbb{O}). \quad (2)$$

3.3.2. Constraint

We first introduce additional notations before presenting the constraints. For a given layer $u \in \mathbb{V}$, $\mathbb{S}_u$ represents its set of intra-layer parallel strategies, A_{uk} denotes the k -th intra-layer execution cost obtained from our time cost model. Additionally, we use $S_{uk} \in \{0, 1\}$ to indicate whether the k -th parallel strategy is selected for the layer u , and use $P_{ui} \in \{0, 1\}$ to indicate whether layer u is placed on the i -th computation stage. Each edge $\langle u, v \rangle \in \mathbb{E}$ is assigned a resharding cost denoted by R_{uv} if the vertices are located within the same pipeline stage. Alternatively, if the vertices are located across consecutive stages, the resharding cost between them is denoted by R'_{uv} . These two resharding costs are constant matrices derived from our time cost model.

Computation-stage constraint To compute the total cost for a single computation stage i , all computation and communication costs associated with that stage must be aggregated and assigned to p_i . This constraint can be formulated as

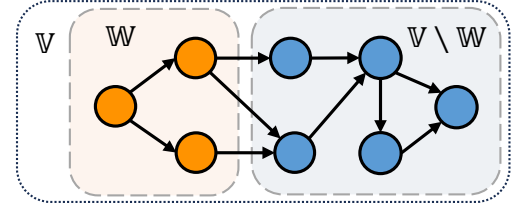


Figure 4. A contiguous set.

follows:

$$\sum_{u \in \mathbb{V}} P_{ui} S_u^T A_u + \sum_{\langle u, v \rangle \in \mathbb{E}} P_{ui} P_{vi} (S_u^T R_{uv} S_v) = p_i, \quad \forall i \in \{1, \dots, deg\}. \quad (3)$$

On the left side of Equation (3), the first polynomial term represents the cost of choosing specific intra-layer strategies for layers placed in stage i . The second term represents total resharding costs within stage i .

Communication-stage constraint To calculate the total cost for a single communication stage j , we should aggregate the P2P costs incurred between consecutive stages and assign them to o_j . This constraint can be formulated as follows:

$$\sum_{\langle u, v \rangle \in \mathbb{E}} P_{uj} P_{v(j+1)} (S_u^T R'_{uv} S_v) = o_j, \quad \forall j \in \{1, \dots, deg-1\}. \quad (4)$$

Memory constraint We need to guarantee that no devices (GPUs) will encounter out-of-memory (OOM) exceptions during training process. This constraint can be formulated as follows:

$$\sum_{u \in \mathbb{V}} P_{ui} S_u^T M_u \leq m, \quad \forall i \in \{1, \dots, deg\}. \quad (5)$$

Here, m denotes the memory limit for each device. In the case of homogeneous computing devices, the value of m remains constant throughout all stages. But the value of m varies in the case of heterogeneous computing devices.

Order-preserving constraint PP is not a single-program multiple-data (SPMD) parallel strategy [15]. Hence, we need an order-preserving constraint to ensure that the subgraphs of $\mathcal{G}$ are contiguous. We adopt the definition of *contiguous* from previous work [38, 39].

Definition 3.1. A set $\mathbb{W} \subseteq \mathbb{V}$ is contiguous if there do not exist nodes $u \in \mathbb{W}$, $v \in \mathbb{V} \setminus \mathbb{W}$, and $w \in \mathbb{W}$ such that v is reachable from u and w is reachable from v .

Figure 4 illustrates an example of a contiguous set $\mathbb{W}$, in which we cannot find any reachable node pairs $\langle u, v \rangle$ and $\langle v, w \rangle$ where $u, w \in \mathbb{W}$ and $v \in \mathbb{V} \setminus \mathbb{W}$.

In our case, our model will not be assigned to different pipeline stages in a disordered manner if we ensure that all subgraphs on each computation stage are contiguous. After reformulating this constraint in linear form, we have

$$\begin{aligned} \mathbf{Z}_{vi} &\geq \mathbf{P}_{vi}, \\ \forall v \in \mathbb{V}, \forall i \in \{1, 2, \dots, \text{deg}\}, \end{aligned} \quad (6a)$$

$$\begin{aligned} \mathbf{Z}_{vi} &\leq \mathbf{Z}_{ui}, \\ \forall u, v \in \mathbb{V}, \forall \langle u, v \rangle \in \mathbb{E}, \forall i \in \{1, 2, \dots, \text{deg}\}, \end{aligned} \quad (6b)$$

$$\begin{aligned} \mathbf{Z}_{vi} &\leq \mathbf{P}_{vi} - \mathbf{P}_{ui} + 1, \\ \forall u, v \in \mathbb{V}, \forall \langle u, v \rangle \in \mathbb{E}, \forall i \in \{1, 2, \dots, \text{deg}\}. \end{aligned} \quad (6c)$$

Here, $\mathbf{Z}$ is an auxiliary variable shaped like $\mathbf{P}$. Given the node set $\mathbb{V}_i$ that contains nodes in stage i , we define $\mathbf{Z}_{vi} = 1$ if a node $w \in \mathbb{V}_i$ is reachable from v ($v \in \mathbb{V}$). Otherwise, $\mathbf{Z}_{vi} = 0$. Detailed proof can be found in Appendix B.

Layer-placement constraint All layers should be placed on exactly one pipeline stage and at least one layer should be placed on each pipeline stage. This constraint can be formulated as follows:

$$\sum_{i=1}^{\text{deg}} \mathbf{P}_{ui} = 1, \quad \forall u \in \mathbb{V}, \quad (7a)$$

$$\sum_{u \in \mathbb{V}} \mathbf{P}_{ui} \geq 1, \quad \forall i \in \{1, \dots, \text{deg}\}, \quad (7b)$$

$$\mathbf{P}_{ui} \in \{0, 1\}, \quad \forall u \in \mathbb{V}, i \in \{1, \dots, \text{deg}\}. \quad (7c)$$

Strategy-selection constraint Each layer must and can choose only one strategy. This constraint can be formulated as follows:

$$\sum_{k=1}^{|\mathbb{S}_u|} \mathbf{S}_{uk} = 1, \quad \forall u \in \mathbb{V}, \quad (8a)$$

$$\mathbf{S}_{uk} \in \{0, 1\}, \quad \forall u \in \mathbb{V}, k \in \{1, \dots, |\mathbb{S}_u|\}. \quad (8b)$$

The MIQP formulation for UniAP includes the objective function in Equation (2) and all the constraints from Equation (3) - (8b).

3.4. Unified optimization process

UOP integrates the cost model and MIQP based on the profiling results and the computation graph to return the optimal parallel strategy and the corresponding TPI. Algorithm 1 summarizes the whole process. In Algorithm 1, we denote intra-layer cost as $\mathbf{A}$, inter-layer cost as $\mathbf{R}$, cross-stage cost as $\mathbf{R}'$, and memory cost as $\mathbf{M}$. The `CostModeling` process calculates these four costs based on the cost model described in Section 3.2.

First, UOP optimizes intra-layer-only parallelism for cases in which pipeline parallelism is not adopted. Several works [23, 47] have used quadratic integer programming (QIP) to optimize intra-layer-only parallel strategy and

Algorithm 1 Unified Optimization Process

Input: Profiling results PR , strategy dictionary SD , mini-batch size B , computation graph $\mathcal{G}$, and the number of GPUs n .

Output: Optimal cost cost^* , pipeline parallel size deg^* , the number of micro-batches c^* , layer placement $\mathbf{P}^*$, and intra-layer strategy $\mathbf{S}^*$.

```

 $\text{deg}^* = 1;$ 
 $c^* = B;$ 
 $\mathbf{A}, \mathbf{R}, \_, \mathbf{M} = \text{CostModeling}(PR, SD[1], \mathcal{G}, B);$ 
 $\text{cost}^*, \mathbf{P}^*, \mathbf{S}^* = \text{QIP}(\mathbf{A}, \mathbf{R}, \mathbf{M});$ 
Get all factors for  $n$  except 1 and insert them to set  $\mathbb{F}$ ;
Get all factors for  $B$  except 1 and insert them to set  $\mathbb{B}$ ;
for  $\text{deg}$  in  $\mathbb{F}$  do
  for  $c$  in  $\mathbb{B}$  do
    Micro-batch size  $b = B/c$ ;
     $\mathbf{A}, \mathbf{R}, \mathbf{R}', \mathbf{M} = \text{CostModeling}(PR, SD[\text{deg}], \mathcal{G}, b);$ 
     $\text{cost}, \mathbf{P}, \mathbf{S} = \text{MIQP}(\mathbf{A}, \mathbf{R}, \mathbf{R}', \mathbf{M}, \text{deg}, c);$ 
    if  $\text{cost} < \text{cost}^*$  then
       $\text{cost}^*, \text{deg}^*, c^*, \mathbf{P}^*, \mathbf{S}^* = \text{cost}, \text{deg}, c, \mathbf{P}, \mathbf{S};$ 
    end if
  end for
end for

```

achieved promising results. UniAP provides a QIP formulation for intra-layer-only parallelism in Appendix C.

Then, UOP enumerates all factors of n except 1 as the pipeline parallel size deg . For each deg , UOP enumerates all factors of B except 1 as the number of micro-batches c . These enumerations aim to achieve load balance on a homogeneous cluster.

For each candidate deg and c , UOP formulates the cost for a training iteration to an MIQP expression. It then waits for the MIQP solver to return the optimal cost and parallel strategy.

Finally, UOP returns the minimum cost cost^* and its corresponding pipeline parallel size deg^* , number of micro-batches c^* , layer placement $\mathbf{P}^*$, and intra-layer strategies $\mathbf{S}^*$. We provide visualization for a candidate solution to UOP in Appendix D.

3.5. Complexity analysis

Let $|\mathbb{V}|$, $|\mathbb{S}|$, and n denote the number of layers, parallel strategies, and GPUs, respectively. As illustrated in Algorithm 1, UniAP enumerates all factors of n except 1 as the deg in the outer loop and all factors of B except 1 in the inner loop. The time complexity for the two loops in the UniAP algorithm is $\mathcal{O}(\sqrt{Bn})$. Within the inner loop body, UniAP calls `CostModeling` to model the cost of each stage for each parallel strategy. Furthermore, the optimization time limit of the MIQP solver can be set as a constant

hyperparameter when UniAP calls it. Therefore, the overall computational complexity of UniAP is $\mathcal{O}(|\mathbb{V}||\mathbb{S}|\sqrt{Bn})$.

4. Experiment

4.1. Experiment setup

We conduct experiments on four different kinds of environments to validate the effectiveness and universality of our method. ENV A refers to a node (machine) with 1 Xeon 6248 CPU, 8 V100-SXM2 32GB GPUs, and 472GB memory. ENV B refers to 4 nodes interconnected with 10Gbps networks and each node has 2 Xeon E5-2620 v4 CPUs, 4 TITAN Xp 12GB GPUs, and 125GB memory. ENV C refers to a node with 8 A100 40GB PCIe GPUs. ENV D is a cloud cluster with 16 nodes and each node has 4 DCUs (a type of non-NVIDIA GPU). In the following text, we specify the number of GPUs after the environment name (e.g., ENV B-8) only when partial nodes of the environment are used.

We evaluate UniAP with five Transformer-based models, BERT-Huge [5], T5-Large [31], ViT-Huge [6], Swin-Huge [24], and Llama [40, 41]. We follow the common practice of training these transformer-based models. To eliminate factors that affect training throughput, we turn off techniques orthogonal to parallel strategies, such as activation checkpointing [3]. However, we integrate FP16 mixed precision training [26] for the largest model, Llama, in some cases to successfully orchestrate the process. More details on these models are provided in Appendix E.

Several solvers can be used for QIP and MIQP optimization. In our implementation, we choose Gurobi [13], with detailed configurations provided in Appendix E. Exploring other solvers is left for future work.

The experimental evaluation concentrates on two primary metrics: training throughput and strategy optimization time. The former is calculated by averaging throughput from the 10th to the 60th iteration of training, while the latter is determined by measuring the time of the UOP. More details are provided in Appendix E. Besides, we provide the accuracy of our cost model in Appendix F.

4.2. Training throughput and strategy optimization time

We compare the training throughput and strategy optimization time of UniAP with those of the baselines on ENV A, ENV B-8, ENV C, and ENV D-32. For experiments on ENV A, we set the mini-batch size to be 32, 16, 128, and 128 for BERT, T5, ViT, and Swin, respectively. For experiments on ENV B-8, we set the mini-batch size to be 16, 8, 64, and 32 for these four models, respectively. For the Llama model [40, 41] run on ENV C, we set the mini-batch size to be 8. For experiments on ENV A, ENV B-8, and ENV C, we choose Galvatron [25] and Alpa [47] as baselines because they have achieved SOTA performance. Specifically,

Galvatron has surpassed other methods, including PyTorch DDP [22], Megatron-LM [30], FSDP [8, 32], GPipe [15], and DeepSpeed 3D [27] in terms of training throughput, as reported in the original paper [25]. Furthermore, Alpa utilizes the Just-In-Time (JIT) compilation feature in JAX and outperforms Megatron-LM and DeepSpeed.

For benchmarks on ENV D-32, pytorch libraries and frameworks need specialized adaptations due to the heterogeneity of DCUs against NVIDIA GPUs. Thus, this part of experiments is based on adapted pytorch libraries and Megatron-DeepSpeed[27] framework provided by the platform developers, and we select MP methods including Megatron [30] and DeepSpeed (ZeRO-3) [34] as our baseline methods. We set the mini-batch size as 8 for Llama-7B and 4 for Llama-13B. The remaining configurations align with those for Llama-7B in Section 4.1, such as activating FP16 mixed precision training.

Table 1 shows the training throughput and strategy optimization time on ENV A, ENV B-8, and ENV C. On ENV A, UniAP and Galvatron get the same optimal strategy for BERT-Huge, T5-Large, and ViT-Huge, outperforming Alpa in terms of training throughput and strategy optimization time. In addition, UniAP finds a solution for Swin-Huge, while Galvatron encounters CUDA OOM issues. In particular, UniAP achieves a maximum optimization speedup that is $17\times$ faster than Galvatron and hundreds of times faster than Alpa on BERT-Huge. This is mainly due to the ability of the MIQP solver to search for an optimal strategy on multiple threads, while the dynamic programming based methods like Galvatron and Alpa run on a single thread due to their strong data dependency.

On ENV B-8, UniAP consistently demonstrates competitive or larger training throughput compared to Galvatron and Alpa. We attribute the performance improvement to UniAP’s larger strategy space. A detailed study is provided in Section 4.4. Furthermore, UniAP’s strategy optimization time is also significantly shorter than the two baseline methods.

On ENV C, UniAP shows an optimization speedup of $11.83\times$ and a training speedup of $3.80\times$ compared to Galvatron on Llama. We examine the parallel strategy they adopted. UniAP employs an 8-stage PP with a micro-batch size of 1, whereas Galvatron employs a 4-stage PP with a micro-batch size of 3 (with the final micro-batch containing 2 samples). Within each PP stage, Galvatron utilizes a 2-way TP. For ENV C, which is equipped with 8 A100 40GB PCIe GPUs, minimizing communication volume is critical. Since PP has significantly less inter-device communication volume than TP, the parallel strategy discovered by UniAP is more reasonable than Galvatron in this case.

Table 2 shows the results on ENV D-32. On this environment, UniAP consistently identifies the fastest parallel strategies while expending considerably less time on strategy optimization compared to Megatron (about $157\times$ faster for

Env.	Model	Training throughput (samples/s)			Minimum speedup	Maximum speedup
		Galvatron	Alpa	UniAP		
ENV A	BERT-Huge	33.46 ± 0.28	31.56 ± 0.04	33.46 ± 0.28	1.00	1.06
	T5-Large	23.29 ± 0.04	MEM× ²⁾	23.29 ± 0.04	1.00	1.00
	ViT-Huge	109.51 ± 0.07	97.66 ± 1.42	109.51 ± 0.07	1.00	1.12
	Swin-Huge	CUDA× ³⁾	N/A ⁴⁾	67.96 ± 0.12	N/A ⁴⁾	N/A ⁴⁾
ENVB-8	BERT-Huge	6.27 ± 0.17	8.95 ± 0.06	10.77 ± 0.13	1.20	1.71
	T5-Large ¹⁾	8.06 ± 0.06	MEM× ²⁾	7.98 ± 0.05	0.99	0.99
	ViT-Huge	32.20 ± 0.17	38.74 ± 0.20	45.58 ± 0.54	1.18	1.41
	Swin-Huge	13.90 ± 0.17	N/A ⁴⁾	19.08 ± 0.10	1.37	1.37
ENVC	Llama-7B	1.22 ± 0.01	N/A ⁴⁾	4.63 ± 0.007	3.80	3.80

Env.	Model	Strategy optimization time (min.)			Minimum speedup	Maximum speedup
		Galvatron	Alpa	UniAP		
ENV A	BERT-Huge	6.44 ± 0.588	> 40	0.37 ± 0.002	17.29	> 107.41
	T5-Large	12.41 ± 0.122	MEM× ²⁾	0.89 ± 0.007	13.98	13.98
	ViT-Huge	6.29 ± 0.464	> 40	0.57 ± 0.009	10.95	> 69.60
	Swin-Huge	11.88 ± 0.666	N/A ⁴⁾	2.16 ± 0.004	5.49	5.49
ENVB-8	BERT-Huge	2.04 ± 0.010	> 40	1.51 ± 0.005	1.34	> 26.32
	T5-Large ¹⁾	2.64 ± 0.110	MEM× ²⁾	0.91 ± 0.005	2.90	2.90
	ViT-Huge	2.37 ± 0.180	> 40	1.11 ± 0.011	2.14	> 36.01
	Swin-Huge	4.29 ± 0.320	N/A ⁴⁾	2.29 ± 0.010	1.87	1.87
ENVC	Llama-7B	6.84 ± 0.055	N/A ⁴⁾	0.58 ± 0.006	11.83	11.83

¹⁾ T5-Large tested on ENVB is restricted to 16/16 layers to avoid out-of-memory (OOM) exceptions.

²⁾ MEM×: OOM exceptions during strategy optimization.

³⁾ CUDA×: CUDA OOM exceptions during model training.

⁴⁾ The official implementation for Swin-Huge and Llama in Alpa is absent. We have endeavored to adopt the code, yet encountered compilation errors. Consequently, experiments related to these models on Alpa are marked as N/A.

Table 1. Training throughput and strategy optimization time on ENV A, ENVB-8, and ENVC.

Model	Training throughput (samples/s)			Strategy optimization time (min.)		
	Megatron	DeepSpeed	UniAP	Megatron	DeepSpeed	UniAP
Llama-7B	2.01 ± 0.005	SOL× ¹⁾	2.01 ± 0.005	> 8.0 hours	SOL× ¹⁾	3.07 ± 0.121
Llama-13B	0.82 ± 0.001	SOL× ¹⁾	0.82 ± 0.001	> 2.5 hours	SOL× ¹⁾	1.95 ± 0.076

¹⁾ SOL×: No solution after strategy optimization.

Table 2. Training throughput and strategy optimization time on ENVD-32.

Llama-7B and 77× faster for Llama-13B). Please note that the strategy optimization time Megatron needs for Llama-7B surpasses that for Llama-13B. We attribute this phenomenon to the variations in the mini-batch size. Specifically, the escalation from a mini-batch size of 4 (employed in Llama-13B) to 8 (employed in Llama-7B) leads to a rise in the number of candidate parallel strategies, as well as the parallel strategies that will successfully train the model. As a result, the strategy optimization time will become increasingly long when the mini-batch size exceeds 8 in the pretraining scenario [1, 40, 41]. In this situation, UniAP will identify the optimal parallel strategy much faster.

Furthermore, our experiments highlight a limitation encountered with DeepSpeed (ZeRO-3). It requires the mini-

batch size to be divisible evenly by the total number of computation devices. This specific prerequisite prevents DeepSpeed from successfully launching the training process with 32 DCUs.

To facilitate further discussions, we provide a case study of the optimal parallel strategy in Appendix G.

4.3. Scalability

We study the scalability of UniAP using BERT, T5, ViT, and Swin on ENVB. We set the mini-batch sizes for each data point as 8, 4, 32, and 16 times the number of nodes (denoted as ‘#nodes’). The experimental results are shown in Figure 5. In Figure 5a, the training throughput of the optimal strategy demonstrate near-linearity as the number of nodes and mini-

Model	Batch size	Training throughput (samples/s)				#infeasible ⁵⁾	#candidate ⁶⁾
		1 st. ¹⁾	2 nd. ²⁾	Slowest ³⁾	Median ⁴⁾		
Llama-7B	8	2.01	1.92	0.22	0.82	41	64
Llama-13B	4	0.82	0.58	0.27	0.42	42	48

¹⁾ 1 st.: The training throughput achieved by the fastest parallel strategy.

²⁾ 2 nd.: The training throughput achieved by the second fastest parallel strategy.

³⁾ Slowest: The training throughput achieved by the slowest parallel strategy.

⁴⁾ Median: The median value of training throughputs across parallel strategies that will successfully train the model.

⁵⁾ #infeasible: The number of parallel strategies that will encounter exceptions such as CUDA OOM during model training.

⁶⁾ #candidate: The total number of candidate parallel strategies.

Table 3. Statistics on the candidate parallel strategies for Megatron.

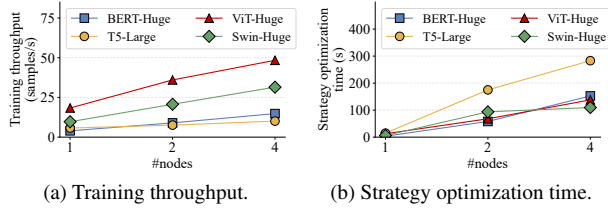


Figure 5. Scalability on training throughput and strategy optimization time.

Number of DCUs	Training throughput (samples/s)	Strategy optimization time(min.)
16	0.1726 ± 0.0005	0.75 ± 0.0100
32	0.3393 ± 0.0004	1.64 ± 0.0321
64	0.6656 ± 0.0023	1.55 ± 0.0058

Table 4. Scalability on training throughput among DCU clusters.

batch size increase. In Figure 5b, the strategy optimization time matches the complexity analysis in Section 3.5.

Additionally, we use DCU nodes in ENV D to scale UniAP to larger clusters on Llama-7B, with precision of FP32. We conducted experiments on clusters with 16, 32, and 64 DCUs, where we set the mini-batch size to 1/8 of the number of DCUs (i.e., for a cluster with 16 DCUs, we set the mini-batch size to 2). The experimental results are shown in Table 4, further demonstrating the near-linear scalability of UniAP.

4.4. Further study on the significance of UniAP

Finally, we investigate the significance of UniAP by answering two questions. The first question is why we need AP rather than MP. We examine the statistics on the candidate parallel strategies for Megatron, as shown in Table 3. With MP, most users are unfamiliar with the complex hyperparameter detail of the candidate parallel strategies and typically choose a parallel strategy at random, which will result in a 64.1% (41/64) chance of failing to train Llama-7B and an 87.5% (42/48) chance of failing to train Llama-13B even when the hardware resources are sufficient. Even if they successfully eliminate all infeasible parallel strategies, they still only have a 50% chance of identifying a parallel strat-

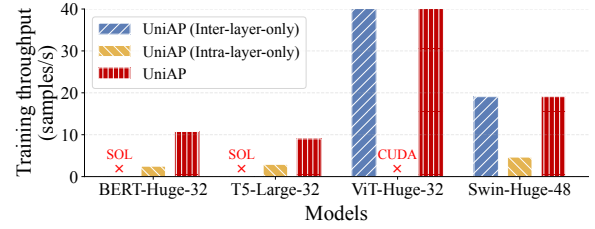


Figure 6. Ablation study. SOL $\times$ indicates no feasible solution during strategy optimization, and CUDA $\times$ indicates CUDA OOM exceptions during training.

egy with training throughput better than 0.82 samples/s for Llama-7B and 0.42 samples/s for Llama-13B. When the users can't identify the fastest parallel strategy out of hundreds of candidates, they may sacrifice at least 4.4% of the training throughput for Llama-7B and 29.2% for Llama-13B. In such circumstances, UniAP is able to identify the fastest strategy, as shown in Table 2.

The second question is why UniAP achieves better performance than other AP methods. We investigate the importance of the strategy space for the optimality of parallel strategies with an ablation study. Specifically, we constrain the strategy space to inter-layer-only and intra-layer-only strategies and evaluate the training throughput of the resulting optimal strategy on ENV B. We set the mini-batch sizes to be 16, 12, 64, and 32, respectively. Results are shown in Figure 6. We can find that constraining the strategy space compromises the optimality of parallel strategies or gets strategies that encounter OOM across different models. Hence, unifying inter- and intra-layer AP for joint optimization makes UniAP outperform other AP methods.

5. Conclusion

In this paper, we propose a novel AP method called UniAP to unify inter- and intra-layer AP by MIQP. To the best of our knowledge, UniAP is the first parallel method that can jointly optimize the two categories of parallel strategies to find an optimal solution. Experimental results show that UniAP can outperform other state-of-the-art baselines to achieve the best performance.

6. Acknowledgement

This work is supported by National Key R&D Program of China (No.2020YFA0713900), NSFC Project (No.12326615), the Key Major Project of the Pengcheng Laboratory (No.PCL2024A06).

References

- [1] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems* 33, pages 1877–1901, 2020. 1, 7
- [2] Zhenkun Cai, Xiao Yan, Kaihao Ma, Yidi Wu, Yuzhen Huang, James Cheng, Teng Su, and Fan Yu. TensorOpt: Exploring the Tradeoffs in Distributed DNN Training With Auto-Parallelism. *IEEE Transactions on Parallel and Distributed Systems*, 33(8):1967–1981, 2022. 1, 2
- [3] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training Deep Nets with Sublinear Memory Cost. *CoRR*, abs/1604.06174, 2016. 6
- [4] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayanan Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. PaLM: Scaling Language Modeling with Pathways. *Journal of Machine Learning Research*, 24:240:1–240:113, 2023. 17
- [5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *The North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4171–4186, 2019. 6, 14
- [6] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *International Conference on Learning Representations*, 2021. 6, 14
- [7] Nan Du, Yanping Huang, Andrew M. Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, Barret Zoph, Liam Fedus, Maarten P. Bosma, Zongwei Zhou, Tao Wang, Yu Emma Wang, Kellie Webster, Marie Pellat, Kevin Robinson, Kathleen S. Meier-Hellstern, Toju Duke, Lucas Dixon, Kun Zhang, Quoc V. Le, Yonghui Wu, Zhifeng Chen, and Claire Cui. GLaM: Efficient Scaling of Language Models with Mixture-of-Experts. In *International Conference on Machine Learning*, pages 5547–5569, 2022. 1
- [8] FairScale authors. FairScale: A general purpose modular PyTorch library for high performance and large scale training. <https://github.com/facebookresearch/fairscale>, 2021. 1, 2, 6, 14
- [9] Shiqing Fan, Yi Rong, Chen Meng, Zongyan Cao, Siyu Wang, Zhen Zheng, Chuan Wu, Guoping Long, Jun Yang, Lixue Xia, Lansong Diao, Xiaoyong Liu, and Wei Lin. DAPPLE: A Pipelined Data Parallel Approach for Training Large Models. In *Symposium on Principles and Practice of Parallel Programming*, pages 431–445, 2021. 1, 2, 3
- [10] William Fedus, Barret Zoph, and Noam Shazeer. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *The Journal of Machine Learning Research*, 23(120):5232–5270, 2022. 1
- [11] Michael J. Flynn. Very High-speed Computing Systems. *Proceedings of the IEEE*, 54(12):1901–1909, 1966. 1
- [12] Michael J. Flynn. Some Computer Organizations and Their Effectiveness. *IEEE Transactions on Computers*, C-21(9):948–960, 1972. 1
- [13] Gurobi Optimization, LLC. Gurobi Optimizer Reference Manual. <https://www.gurobi.com>, 2023. 6, 14
- [14] Chaoyang He, Shen Li, Mahdi Soltanolkotabi, and Salman Avestimehr. PipeTransformer: Automated Elastic Pipelining for Distributed Training of Large-scale Models. In *International Conference on Machine Learning*, pages 4150–4159, 2021. 1, 2
- [15] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Xu Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism. In *Advances in Neural Information Processing Systems* 32, pages 103–112, 2019. 1, 2, 4, 6, 14
- [16] Zhihao Jia, Sina Lin, Charles R. Qi, and Alex Aiken. Exploring Hidden Dimensions in Parallelizing Convolutional Neural Networks. In *International Conference on Machine Learning*, pages 2279–2288, 2018. 1, 2
- [17] Zhihao Jia, Matei Zaharia, and Alex Aiken. Beyond Data and Model Parallelism for Deep Neural Networks. In *Machine Learning and Systems*, pages 1–13, 2019. 1, 2
- [18] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations*, 2015. 14

- [19] Rafael Lazimy. Mixed-integer Quadratic Programming. *Math. Program.*, 22(1):332–349, 1982. 1
- [20] Dmitry Lepikhin, Hyoungho Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. In *International Conference on Learning Representations*, 2021. 1
- [21] Shigang Li and Torsten Hoefler. Chimera: Efficiently Training Large-Scale Neural Networks with Bidirectional Pipelines. In *International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 27:1–27:14, 2021. 1, 3
- [22] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, and Soumith Chintala. PyTorch Distributed: Experiences on Accelerating Data Parallel Training. *Proceedings of the VLDB Endowment*, 13(12):3005–3018, 2020. 1, 2, 6, 14
- [23] Yuliang Liu, Shenggui Li, Jiarui Fang, Yanjun Shao, Boyuan Yao, and Yang You. Colossal-Auto: Unified Automation of Parallelization and Activation Checkpoint for Large-scale Models. *CoRR*, abs/2302.02599, 2023. 1, 2, 5
- [24] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin Transformer: Hierarchical Vision Transformer using Shifted Windows. In *International Conference on Computer Vision*, pages 9992–10002, 2021. 6, 14
- [25] Xupeng Miao, Yujie Wang, Youhe Jiang, Chunan Shi, Xiaonan Nie, Hailin Zhang, and Bin Cui. Galvatron: Efficient Transformer Training over Multiple GPUs Using Automatic Parallelism. *Proceedings of the VLDB Endowment*, 16(3): 470–479, 2022. 2, 3, 6, 15
- [26] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory F. Diamos, Erich Elsen, David García, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed Precision Training. In *International Conference on Learning Representations*, 2018. 6
- [27] Microsoft. Megatron-DeepSpeed. <https://github.com/microsoft/Megatron-DeepSpeed/tree/main>, 2021. 6
- [28] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei Zaharia. PipeDream: Generalized Pipeline Parallelism for DNN Training. In *Symposium on Operating Systems Principles*, pages 1–15, 2019. 1, 2
- [29] Deepak Narayanan, Amar Phanishayee, Kaiyu Shi, Xie Chen, and Matei Zaharia. Memory-Efficient Pipeline-Parallel DNN Training. In *International Conference on Machine Learning*, pages 7937–7947, 2021. 1, 3
- [30] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. Efficient Large-scale Language Model Training on GPU Clusters Using Megatron-LM. In *International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 58:1–58:15, 2021. 1, 2, 3, 6, 14
- [31] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research*, 21(1):5485–5551, 2020. 6, 14
- [32] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. In *International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 20:1–20:16, 2020. 2, 6
- [33] Saeed Rashidi, Matthew Denton, Srinivas Sridharan, Sudarshan Srinivasan, Amoghavarsha Suresh, Jade Nie, and Tushar Krishna. Enabling Compute-Communication Overlap in Distributed Deep Learning Training Platforms. In *International Symposium on Computer Architecture*, pages 540–553, 2021. 3
- [34] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. In *The 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 3505–3506, 2020. 1, 6
- [35] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision*, 115(3):211–252, 2015. 14
- [36] Michael Schaarschmidt, Dominik Grewe, Dimitrios Vytiniotis, Adam Paszke, Georg Stefan Schmid, Tamara Norman, James Molloy, Jonathan Godwin, Norman Alexander Rink, Vinod Nair, and Dan Belov. Automap: Towards Ergonomic Automated Parallelism for ML Models. *CoRR*, abs/2112.02958, 2021. 1, 2
- [37] Noam Shazeer, Youlong Cheng, Niki Parmar, Dustin Tran, Ashish Vaswani, Penporn Koanantakool, Peter Hawkins, Hyoungho Lee, Mingsheng Hong, Cliff Young, Ryan Sepassi, and Blake A. Hechtman. Mesh-TensorFlow: Deep Learning for Supercomputers. In *Advances in Neural Information Processing Systems 31*, pages 10435–10444, 2018. 1, 2
- [38] Jakub Tarnawski, Amar Phanishayee, Nikhil R. Devanur, Divya Mahajan, and Fanny Nina Paravecino. Efficient Algorithms for Device Placement of DNN Graph Operators. In *Advances in Neural Information Processing Systems 33*, pages 15451–15463, 2020. 1, 2, 4, 12
- [39] Jakub Tarnawski, Deepak Narayanan, and Amar Phanishayee. Piper: Multidimensional Planner for DNN Parallelization. In *Advances in Neural Information Processing Systems 34*, pages 24829–24840, 2021. 1, 2, 4
- [40] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurélien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and Efficient Foundation Language Models. *CoRR*, abs/2302.13971, 2023. 1, 6, 7, 14
- [41] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas

- Blecher, Cristian Canton-Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenxin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurélien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open Foundation and Fine-Tuned Chat Models. *CoRR*, abs/2307.09288, 2023. 1, 6, 7, 14
- [42] Minjie Wang, Chien-Chin Huang, and Jinyang Li. Supporting Very Large Models using Automatic Dataflow Graph Partitioning. In *Proceedings of the Fourteenth EuroSys Conference*, pages 26:1–26:17, 2019. 1, 2
- [43] Wikimedia Foundation. Wikimedia Downloads. <https://dumps.wikimedia.org>, 2023. 14
- [44] Yuanzhong Xu, HyoukJoong Lee, Dehao Chen, Blake A. Hechtman, Yanping Huang, Rahul Joshi, Maxim Krikun, Dmitry Lepikhin, Andy Ly, Marcello Maggioni, Ruoming Pang, Noam Shazeer, Shibo Wang, Tao Wang, Yonghui Wu, and Zhifeng Chen. GSPMD: General and Scalable Parallelization for ML Computation Graphs. *CoRR*, abs/2105.04663, 2021. 1, 2
- [45] Binhang Yuan, Yongjun He, Jared Davis, Tianyi Zhang, Tri Dao, Beidi Chen, Percy Liang, Christopher Ré, and Ce Zhang. Decentralized Training of Foundation Models in Heterogeneous Environments. In *Advances in Neural Information Processing Systems 35*, pages 25464–25477, 2022. 2
- [46] Shixiong Zhao, Fanxin Li, Xusheng Chen, Xiuxian Guan, Jianyu Jiang, Dong Huang, Yuhao Qing, Sen Wang, Peng Wang, Gong Zhang, Cheng Li, Ping Luo, and Heming Cui. vPipe: A Virtualized Acceleration System for Achieving Efficient and Scalable Pipeline Parallel DNN Training. *IEEE Transactions on Parallel and Distributed Systems*, 33(3):489–506, 2022. 1, 2
- [47] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P. Xing, Joseph E. Gonzalez, and Ion Stoica. Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning. In *USENIX Symposium on Operating Systems Design and Implementation*, pages 559–578, 2022. 1, 2, 5, 6